

Build Task — Test Suite for Project 4 — Hints & Solution

[Back to the Build Task](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Only 2 hints — work through them in order, and don't jump ahead until you've genuinely tried. Each hint has 2 depth levels: **Basic** (the plain idea) and **Intermediate** (the full suite, split into the suggested files). Read Basic first even if you already know the pattern — it's the fastest way to spot exactly what Intermediate adds.

- [Hint 1 — What you're building, and the exact pieces](#)
- [Hint 2 — The plan, and almost the whole thing](#)
- [Solution](#)

[Back to the Build Task](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Hint 1 — What you're building, and the exact pieces

Basic Version

You're building three pieces that work together: a fixed list of test tasks, a judge that scores an answer against rules, and a runner that goes through every task and reports the result. Then you prove the whole thing works by making a prompt worse on purpose.

Nothing here is new — every piece is one of this document's exercises:

- **Exact tests with no model** → `chunking_unit_test` and `tool_test_no_llm`.
- **A judge, and a check that the judge can be trusted** → `llm_judge_scoring`.
- **Checking a short phrase, never the whole answer** → `flaky_test_fix`.
- **Breaking the prompt on purpose and finding which tasks caught it** → `regression_catch`.

Start with the task list. Don't write any code yet. Write down 10 real tasks you'd give Project 4, and for each one, how you'd know the answer was right — either one fact that must appear, or a few rules about meaning.

Intermediate Version

Keep the pieces in separate files, one job each:

1. **tasks.py** — at least 10 tasks, written *before* you run anything. A task has either `"expected"` (one short phrase that must appear — a free code check) or `"rules"` (checked by the judge).
2. **pipeline.py** — `run_project_4(task_input)`. In this solution it's a small stand-in: one model call with the writer's prompt read from `writer_prompt.txt`. Swap its body for your real Project 4 graph. Reading the prompt from a file on every call is what lets the regression check edit it mid-run, like `regression_catch`.
3. **judge.py** — `llm_judge_scoring`'s Approach 1 (typed `JudgeResult`, saved `JUDGE_PROMPT_VERSION`, `temperature=0`), unchanged.
4. **run_eval.py** — loops over `TASKS`: runs Project 4, uses a code check or the judge, prints a summary with the reason for each failure, and saves `report.json`.
5. **regression_check.py** — `regression_catch`'s pass bar plus per-task comparison, pointed at `run_eval_suite()`.
6. **test_unit_layer.py** — plain pytest tests for the chunker and the tools, like `chunking_unit_test` and `tool_test_no_llm`. No model, no API key.

7. `test_judge.py` — `llm_judge_scoring`'s calibration set: the judge must FAIL an obviously bad answer.

The one design choice worth making on purpose: give the writer one rule that code *and* the judge can both see break — here, "end every brief with a line that starts with `In short:`". Every judge task lists that rule, so deleting it from `writer_prompt.txt` must drop the score.

Difference between Basic and Intermediate: Basic names the three pieces and maps each back to an exercise. Intermediate splits them into files with one job each, and plans the regression check from the start — a prompt read from a file, and a rule the tasks can see break.

[Back to the Build Task](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Hint 2 — The plan, and almost the whole thing

Basic Version

```
one file, eval_all.py:
TASKS: a list of dicts – id, input, and "expected" or "rules"
run_project_4(task_input): writer prompt from a file + one model call
score_with_judge(task_input, output, rules): PASS/FAIL from a judge call

for each task:
    output = run_project_4(task input)
    if the task has "expected": passed = expected phrase is in output
    else: passed = the judge says PASS
    print the task id and PASS/FAIL
print how many passed
```

Here's the trickiest part — choosing a code check or the judge for each task:

```
for task in TASKS:
    output = run_project_4(task["input"])
    if "expected" in task:
        passed = task["expected"].lower() in output.lower()
    else:
        passed = score_with_judge(task["input"], output, task["rules"])
    # your turn: print the result, and count how many passed
    ...
```

Intermediate Version

tasks.py	TASKS – 4 "expected" tasks, 6 "rules" tasks
pipeline.py	run_project_4(task_input) – reads writer_prompt.txt each call
judge.py	JudgeResult, JUDGE_PROMPT_VERSION, call_model, build_judge_prompt, score_with_judge
run_eval.py	run_one_task(task) -> {task_id, passed, reason} run_eval_suite() -> {task_id: passed}, saves report.json
regression_check.py	baseline = run_eval_suite() pause: delete the "In short:" line, press Enter

```

        after = run_eval_suite()
        pass bar + newly failed tasks
test_unit_layer.py  chunker + tools, no model
test_judge.py      calibration set

```

Here's `run_one_task()` — write `run_eval_suite()` around it yourself:

```

def run_one_task(task: dict) -> dict:
    output = run_project_4(task["input"])
    if "expected" in task:
        passed = task["expected"].lower() in output.lower()
        reason = "looked for: " + task["expected"]
    else:
        judged = score_with_judge(task["input"], output, task["rules"])
        passed = judged.verdict == "PASS"
        reason = judged.reason
    return {"task_id": task["id"], "passed": passed, "reason": reason}

```

[Back to the Build Task](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Solution

`run_project_4()` below is a small stand-in for Project 4's real multi-agent graph — one model call with the writer's prompt. Everything around it (tasks, judge, runner, regression check, unit tests) is what this Build Task is about, and doesn't change once you swap the real graph in. Every file lives in `practice/build_task/` and runs from inside that folder, with `OPENAI_API_KEY` in your `.env`.

All versions use this prompt file:

```
# practice/build_task/writer_prompt.txt
You are the writer agent in a content team.
Write a short, factual brief (under 120 words) on the topic you are given.
Do not invent statistics or sources.
End every brief with a final line that starts with "In short:".
```

Read both depths — they're not "wrong, right," they're 2 real, valid ways to build the same suite, with real tradeoffs between them.

Basic Version

Approach 1 — one file: tasks, judge and runner together

Story: before splitting anything, get the whole loop — task in, Project 4 answer, code check or judge, PASS/FAIL out — working in one file you can read top to bottom. **If not:** a bug could be in any of five files at once, and you'd debug the layout instead of the logic.

```
# practice/build_task/eval_all.py
from openai import OpenAI

client = OpenAI()

TASKS = [
    {"id": "boiling_point",
     "input": "Write a brief on the boiling point of water at sea level.",
     "expected": "100"},
    {"id": "tea_history",
     "input": "Write a brief on where tea drinking began.",
     "rules": ["says tea drinking began in China",
              'ends with a final line that starts with "In short:"']},
```

```

    # ... 8 more tasks, mixing "expected" and "rules"
]

def call_model(system_prompt, user_text):
    messages = []
    if system_prompt != "":
        messages.append({"role": "system", "content": system_prompt})
    messages.append({"role": "user", "content": user_text})
    response = client.chat.completions.create(
        model="gpt-4o-mini", temperature=0, messages=messages
    )
    return response.choices[0].message.content

def run_project_4(task_input):
    # stand-in for your real Project 4 graph
    with open("writer_prompt.txt") as f:
        writer_prompt = f.read()
    return call_model(writer_prompt, task_input)

def score_with_judge(task_input, output, rules):
    rules_text = ""
    for rule in rules:
        rules_text = rules_text + "- " + rule + "\n"
    prompt = (
        "Task: " + task_input + "\n"
        "Answer to check: " + output + "\n"
        "Rules the answer must follow:\n" + rules_text +
        "Reply with PASS or FAIL on the first line, "
        "then one sentence explaining why."
    )
    reply = call_model("", prompt)
    return reply.strip().split("\n")[0].strip().upper() == "PASS"

passed_count = 0
for task in TASKS:
    output = run_project_4(task["input"])
    if "expected" in task:
        passed = task["expected"].lower() in output.lower()
    else:
        passed = score_with_judge(task["input"], output, task["rules"])
    if passed:
        print(task["id"], "PASS")
        passed_count = passed_count + 1

```

```
    else:
        print(task["id"], "FAIL")

print("Passed", passed_count, "of", len(TASKS))
```

Expected output (with all 10 tasks filled in; which judge tasks pass can vary slightly between runs):

```
boiling_point PASS
tea_history PASS
...
Passed 10 of 10
```

This proves the loop works. It's still missing the unit layer, a saved report, the reason for each failure, a check of the judge itself, and the regression check — all below.

[Back to the Build Task](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Intermediate Version

Approach 1 — the suite, split into the suggested files

Story — `tasks.py`: the fixed test set, written before anything runs. Facts get a free "expected" check; meaning gets "rules" for the judge. **If not:** you'd write rules after reading the outputs, and they'd describe whatever the model said instead of what's correct.

```
# practice/build_task/tasks.py
# why: written BEFORE running anything against Project 4, so the
# rules describe a correct answer, not whatever the model said
IN_SHORT_RULE = 'ends with a final line that starts with "In short:"'

TASKS = [
    # exact-check tasks: one fact that must appear, word for word
    {"id": "boiling_point",
     "input": "Write a brief on the boiling point of water at sea level.",
     "expected": "100"},
    {"id": "france_capital",
     "input": "Write a brief on the capital city of France.",
     "expected": "Paris"},
    {"id": "ww2_end",
     "input": "Write a brief on the year World War II ended.",
     "expected": "1945"},
    {"id": "water_formula",
     "input": "Write a brief on the chemical formula of water.",
     "expected": "H2O"},
    # judge tasks: rules about meaning, checked by judge.py
    {"id": "sleep_students",
     "input": "Write a brief on why sleep matters for students.",
     "rules": ["explains at least one real effect of sleep on learning",
              "does not invent statistics", IN_SHORT_RULE]},
    {"id": "remote_work",
     "input": "Write a brief on one benefit and one risk of remote work.",
     "rules": ["names one benefit and one risk", IN_SHORT_RULE]},
    {"id": "password_safety",
     "input": "Write a brief on how to choose a strong password.",
     "rules": ["gives at least two practical tips", IN_SHORT_RULE]},
    {"id": "solar_power",
     "input": "Write a brief on how solar panels make electricity.",
     "rules": ["explains that sunlight is turned into electricity",
              "does not invent statistics", IN_SHORT_RULE]},
```

```
{
  "id": "tea_history",
  "input": "Write a brief on where tea drinking began.",
  "rules": ["says tea drinking began in China", IN_SHORT_RULE]},
{
  "id": "recycling",
  "input": "Write a brief on why recycling plastic is hard.",
  "rules": ["gives at least one real reason", IN_SHORT_RULE]},
}
```

Story — `pipeline.py` : the one door into Project 4. It reads the writer prompt fresh on every call, so the regression check can edit the file mid-run. **If not:** every file would call Project 4 its own way, and swapping in the real graph would mean editing all of them.

```
# practice/build_task/pipeline.py
from openai import OpenAI

client = OpenAI()

def load_writer_prompt() -> str:
    # why: read fresh on every call, so an edit to the file shows up
    # on the next run – this is what the regression check sabotages
    with open("writer_prompt.txt") as f:
        return f.read()

def run_project_4(task_input: str) -> str:
    # stand-in for your real Project 4 graph – swap this body for
    # something like: graph.invoke({"task": task_input})["draft"]
    response = client.chat.completions.create(
        model="gpt-4o-mini",
        temperature=0,
        messages=[
            {"role": "system", "content": load_writer_prompt()},
            {"role": "user", "content": task_input},
        ],
    )
    return response.choices[0].message.content
```

Story — `judge.py` : `llm_judge_scoring` 's Approach 1, unchanged — a typed result, a versioned prompt, `temperature=0` . **If not:** the Build Task would bring in a second, different judge, with no calibration behind it.

```
# practice/build_task/judge.py
from dataclasses import dataclass
from openai import OpenAI
```

```

client = OpenAI()

# why: every score is tied to one exact judge prompt – change the
# prompt, change this name
JUDGE_PROMPT_VERSION = "v1"

@dataclass
class JudgeResult:
    verdict: str
    reason: str

def call_model(prompt: str) -> str:
    # how: temperature=0 – a judge should be steady, not creative
    response = client.chat.completions.create(
        model="gpt-4o-mini",
        temperature=0,
        messages=[{"role": "user", "content": prompt}],
    )
    return response.choices[0].message.content

def build_judge_prompt(task_input: str, output: str, rules: list) -> str:
    rules_text = ""
    for rule in rules:
        rules_text = rules_text + "- " + rule + "\n"
    return (
        "Task: " + task_input + "\n"
        "Answer to check: " + output + "\n"
        "Rules the answer must follow:\n" + rules_text +
        "Reply with PASS or FAIL on the first line, "
        "then one sentence explaining why."
    )

def score_with_judge(task_input: str, output: str, rules: list):
    reply = call_model(build_judge_prompt(task_input, output, rules))
    lines = reply.strip().split("\n", 1)
    verdict = lines[0].strip().upper()
    if len(lines) > 1:
        reason = lines[1].strip()
    else:
        reason = ""
    return JudgeResult(verdict=verdict, reason=reason)

```

Story — `run_eval.py` : runs every task, uses the cheapest check that works, and leaves a `report.json` behind. **If not:** a run's result would live only in your terminal, and a failing run wouldn't say *why*.

```
# practice/build_task/run_eval.py
import json

from judge import JUDGE_PROMPT_VERSION, score_with_judge
from pipeline import run_project_4
from tasks import TASKS

def run_one_task(task: dict) -> dict:
    output = run_project_4(task["input"])
    # when: a task with "expected" gets a free code check;
    # only tasks with "rules" cost a judge call
    if "expected" in task:
        passed = task["expected"].lower() in output.lower()
        reason = "looked for: " + task["expected"]
    else:
        judged = score_with_judge(task["input"], output, task["rules"])
        passed = judged.verdict == "PASS"
        reason = judged.reason
    return {"task_id": task["id"], "passed": passed, "reason": reason}

def run_eval_suite() -> dict:
    results = []
    passed_by_id = {}
    for task in TASKS:
        result = run_one_task(task)
        results.append(result)
        passed_by_id[result["task_id"]] = result["passed"]

    passed_count = 0
    for result in results:
        if result["passed"]:
            passed_count = passed_count + 1
    overall_score = passed_count / len(results)

    # why: a saved record of this run, with the judge version,
    # so a score can be checked later without re-running anything
    report = {
        "judge_prompt_version": JUDGE_PROMPT_VERSION,
        "overall_score": overall_score,
        "passed": passed_count,
        "total": len(results),
    }
```

```

        "results": results,
    }
    with open("report.json", "w") as f:
        json.dump(report, f, indent=2)

    print(f"Passed {passed_count}/{len(results)} "
          f"(score: {overall_score:.2f})")
    for result in results:
        if not result["passed"]:
            print("  FAILED " + result["task_id"] + ": " + result["reason"])

    return passed_by_id

if __name__ == "__main__":
    run_eval_suite()

```

Expected output (`python run_eval.py`):

```
Passed 10/10 (score: 1.00)
```

Story — `test_unit_layer.py` : the fast, free layer, reused from `chunking_unit_test` and `tool_test_no_llm` . **If not:** a broken chunker or tool would only show up as a vague drop in the eval score, with nothing pointing at the function.

```

# practice/build_task/test_unit_layer.py
# No model, no API key, no network – runs in well under a second.
import pytest

from chunking import chunk_by_paragraph
from tools import add, flaky_lookup

def test_chunk_by_paragraph():
    text = "Paragraph one.\n\nParagraph two.\n\nParagraph three."
    chunks = chunk_by_paragraph(text)
    assert len(chunks) == 3
    assert chunks[0] == "Paragraph one."
    assert chunks[-1] == "Paragraph three."

@pytest.mark.parametrize(
    "text,expected_count",
    [
        ("Just one paragraph, no blank lines.", 1),
    ]
)

```

```

        ("A.\n\n\n\nB.", 2),
        ("", 0),
    ],
)
def test_chunk_by_paragraph_count(text, expected_count):
    assert len(chunk_by_paragraph(text)) == expected_count

def test_add_returns_the_sum():
    assert add.invoke({"a": 2, "b": 3}) == 5
    assert add.invoke({"a": -1, "b": 1}) == 0

def test_flaky_lookup_success():
    result = flaky_lookup.invoke({"query": "refund policy"})
    assert result == "Result for refund policy"

def test_flaky_lookup_failure():
    result = flaky_lookup.invoke(
        {"query": "refund policy", "should_fail": True}
    )
    assert result == "Error: simulated failure"

```

Copy Doc08's `chunking.py` and Doc06's `tools.py` (with `http_client.py`, `exceptions.py` and `logging_setup.py`) into `practice/build_task/` first, unchanged.

Expected output (`pytest test_unit_layer.py -q`):

```

..... [100%]
7 passed in 0.15s

```

Break one tool on purpose — change `add` in `tools.py` to return `a - b` — and exactly one test fails, pointing at that function:

```

FAILED test_unit_layer.py::test_add_returns_the_sum - AssertionError: ...
1 failed, 6 passed in 0.24s

```

Approach 2 — prove the judge and the suite, on purpose

Story — `test_judge.py`: `llm_judge_scoring`'s calibration set, pointed at this suite's judge. It must PASS a good brief and FAIL an off-topic one and a confidently wrong one. **If not:** a judge that approves everything would make every run look perfect.

```
# practice/build_task/test_judge.py
# Calls the real judge – run it whenever judge.py's prompt changes.
from judge import score_with_judge

TASK_INPUT = "Write a brief on where tea drinking began."
RULES = [
    "says tea drinking began in China",
    'ends with a final line that starts with "In short:"',
]

# why: answers where YOU already know the right verdict
CALIBRATION_SET = [
    {"output": "Tea drinking began in ancient China, where it was "
              "first used as a medicine.\nIn short: tea started "
              "in China.",
     "expected_verdict": "PASS"},
    {"output": "I like pizza.",
     "expected_verdict": "FAIL"},
    {"output": "Tea drinking began in Brazil in the 1900s.\n"
              "In short: tea is Brazilian.",
     "expected_verdict": "FAIL"},
]

def test_judge_calibration():
    for case in CALIBRATION_SET:
        result = score_with_judge(TASK_INPUT, case["output"], RULES)
        assert result.verdict == case["expected_verdict"], result.reason
```

Expected output (`pytest test_judge.py -v`):

```
test_judge.py::test_judge_calibration PASSED
```

Story — `regression_check.py` : `regression_check` 's pass bar and per-task list, run against the real suite. **If not:** "the suite would catch a regression" would stay a belief, never a result.

```
# practice/build_task/regression_check.py
from run_eval import run_eval_suite

# why: decided BEFORE sabotaging, so the result can't be
# explained away afterwards
PASS_BAR = 0.8

def score(passed_by_id: dict) -> float:
```

```

passed = 0
for task_id in passed_by_id:
    if passed_by_id[task_id]:
        passed = passed + 1
return passed / len(passed_by_id)

def find_newly_failed(baseline: dict, after: dict) -> list:
    newly_failed = []
    for task_id in baseline:
        if baseline[task_id] and not after[task_id]:
            newly_failed.append(task_id)
    return newly_failed

def run_regression_check() -> None:
    print("--- baseline run ---")
    baseline = run_eval_suite()
    input('Delete the "In short:" line from writer_prompt.txt, save, '
          "then press Enter...")
    print("--- after sabotage ---")
    after = run_eval_suite()

    newly_failed = find_newly_failed(baseline, after)
    print(f"score: {score(baseline):.2f} -> {score(after):.2f}")
    if score(baseline) >= PASS_BAR and score(after) < PASS_BAR:
        print("PASS: the suite caught it. Newly failing:")
        for task_id in newly_failed:
            print(" " + task_id)
    else:
        print("FAIL: the suite did NOT catch it. Tighten the rules.")

if __name__ == "__main__":
    run_regression_check()

```

Expected output (`python regression_check.py` ; the second run's **FAILED lines are shortened here):**

```

--- baseline run ---
Passed 10/10 (score: 1.00)
Delete the "In short:" line from writer_prompt.txt, save, then press Enter...
--- after sabotage ---
Passed 4/10 (score: 0.40)
  FAILED sleep_students: The brief does not end with an "In short:" line.
...

```

```
score: 1.00 -> 0.40
PASS: the suite caught it. Newly failing:
    sleep_students
    remote_work
    password_safety
    solar_power
    tea_history
    recycling
```

The 4 fact tasks still pass — the sabotage didn't touch facts — and all 6 judge tasks fail on the one rule that was removed. Put the line back in `writer_prompt.txt` when you're done.

Difference from Basic: Approach 1 splits the single file into one job per file, adds the reason for each failure and a saved `report.json`, and adds the unit layer that points at the exact broken function. Approach 2 proves the two things a green run can't: that the judge fails obviously bad answers, and that the whole suite drops when the prompt gets worse.

Which one should you actually write? Basic Approach 1 first, to see the loop work end to end. Then Intermediate Approach 1 and 2 together — that's what the Requirements ask for. Run `pytest test_unit_layer.py` on every change (it's free), `python run_eval.py` before every prompt or model change, `test_judge.py` whenever `judge.py`'s prompt changes, and `regression_check.py` once per new rule you add — to prove the suite can see it break.